Automated Financial Analytics and Credit Risk Reporting

A reproducible pipeline for P&L, budget variance and scorecard-based risk banding (DA-04)

Mayank Swaraj | Mayank Creations | Nexvia Technologies Capstone | October 2026

Summary

• **Problem:** manual month-end finance and credit-risk reporting is slow, error-prone and hard to audit.

• **Approach:** one reproducible pipeline producing P&L, budget-vs-actual with exception flags, a logistic scorecard (300-850 points, four bands) and an auto-written board pack, delivered as an offline HTML dashboard.

• **Results:** FY2025 revenue INR 10,485,397 (-1.4% vs budget); net income -19.8% vs budget; 2 accounts flagged, matching both injected issues; scorecard hold-out AUC 0.771; 6 of 6 tests pass.

• **Caveat:** data are synthetic, so AUC validates the implementation rather than real-world predictive power.

1. Introduction

Month-end finance reporting in small and mid-sized firms is usually assembled by hand from ledger exports and spreadsheets. The process is slow, hard to audit and prone to copy errors, and the credit-risk view of the customer book is often produced separately, if at all. This paper describes a compact, fully reproducible system that automates four outputs from one pipeline: a monthly profit and loss (P&L) statement, a budget-versus-actual variance analysis with automatic exception flags, a scorecard-based credit risk rating for every customer, and a board-pack narrative that summarises the position in plain language.

The contributions are: (i) an end-to-end pipeline, from data to an offline dashboard, that can be regenerated with one command; (ii) a transparent logistic-regression scorecard that converts probability of default (PD) into a 300 to 850 points scale with named risk bands; (iii) a rule-based variance engine whose flags were verified against deliberately injected issues; and (iv) an automated test suite that checks accounting identities, determinism, model quality and flag behaviour.

2. Background and Related Work

Credit scorecards built on logistic regression remain a standard industry tool because the coefficients are directly interpretable and the points scale is easy to explain to credit officers and auditors [1]. Model discrimination is commonly summarised by the area under the ROC curve (AUC), which equals the probability that a randomly chosen defaulter receives a higher PD than a randomly chosen non-defaulter [2]. Points-to-double-odds (PDO) scaling, used here, maps log-odds linearly to points so that a fixed number of points always corresponds to a doubling of good-to-bad odds.

Budget variance analysis compares actual results with plan for each account and reports the difference by sign convention: for revenue, actual above budget is favourable; for costs, actual below budget is favourable. Automating the rule is simple, but its value depends on a sensible exception threshold and on correct handling of the sign convention, both of which are made explicit and tested in this work.

3. System Design

The pipeline (pipeline.py) has five stages: data generation, P&L computation, variance analysis, scorecard fitting and scoring, and report generation. Outputs are two CSV files (gl_ledger.csv, customers_scored.csv), a metrics file, and a single self-contained HTML dashboard that embeds its data and draws charts with canvas, so it opens offline in any browser. A PostgreSQL schema (schema.sql) is provided for loading the CSVs into a database.

Table 1. Data model.

Table	Columns
gl_ledger	month, account, type (Revenue/COGS/Opex), budget, actual
customers_scored	id, income, debt_ratio, late_payments, utilization, tenure_years, defaulted, pd, score, band

4. Methods

4.1 Ledger simulation

No real company ledger was available, so a synthetic general ledger was generated with a fixed random seed. Nine accounts across three types (three revenue, one cost of goods sold, five operating expense) are simulated for the twelve months of 2025. The monthly budget for each account is its base value grown by 1% per month. Actuals equal budget multiplied by a seasonal factor (applied to revenue and COGS) and Gaussian noise:

```
actual = budget * season * (1 + e),  season = 1 + 0.12 * sin(2*pi*(i-2)/12),  e ~ N(0, 0.04)
```

Two issues were injected on purpose so that detection can be verified rather than assumed: Marketing spend is raised by 25% in July to September, and Services revenue is reduced by 12% from July to December.

4.2 P&L and variance rules

For every month, gross profit = revenue - COGS and net income = gross profit - operating expenses, for both actual and budget. For each account the favourable variance is (actual - budget) for revenue and (budget - actual) for costs, expressed as a percentage of budget. An account is flagged "Over budget" (cost) or "Below target" (revenue) when the favourable variance is below -5%.

4.3 Credit scorecard

A population of 1,000 customers is simulated with five features: income (log-normal), debt ratio (Beta(2,5)), late payments (Poisson(1.2)), credit utilisation (Beta(2,3)) and tenure in years (uniform 1 to 14). Defaults are drawn from a Bernoulli distribution with probability sigmoid(z), where

```
z = -3.2 + 3.0*debt_ratio + 0.45*late_payments + 1.8*utilization - 0.08*tenure - 0.00001*income
```

The data are split 70/30 by a seeded permutation. Features are standardised using the training set and an L2-regularised logistic regression (scikit-learn defaults, 1,000 iterations) is fitted. AUC is computed on the held-out 30%. Every customer is then scored in-sample for the dashboard using the PDO transformation

```
score = 650 + (40 / ln 2) * ln( (1 - PD) / PD / 20 ),  clipped to [300, 850]
```

which gives 650 points at 20:1 good-to-bad odds and 40 points per doubling of odds. Bands are Low (640 and above), Medium (600 to 639), High (560 to 599) and Very High (below 560); the cut-offs were chosen after inspecting the score distribution so that all four bands are populated.

4.4 Board-pack narrative

A short narrative is generated from the computed figures: full-year revenue and net income against budget, the three worst flagged accounts, and a credit-book summary. Because every sentence is produced from computed values, the text cannot drift from the numbers.

5. Results

Full-year revenue was INR 10,485,397 against a budget of INR 10,634,400 (-1.4%). Net income was INR 984,823 against INR 1,228,020 (-19.8%).

Table 2. FY2025 P&L, actual versus budget.

Line	Actual (INR)	Budget (INR)	Variance
Revenue	10,485,397	10,634,400	-1.4%
COGS	4,877,106	4,810,800	+1.4%
Gross profit	5,608,291	5,823,600	-3.7%
Operating expenses	4,623,469	4,595,580	+0.6%
Net income	984,823	1,228,020	-19.8%

The variance engine flagged 2 account(s): Services (-6.0%, below target); Marketing (-6.0%, over budget). These are exactly the two accounts into which issues were injected, and no other account crossed the 5% threshold.

Table 3. Budget versus actual by account (full year).

Account	Type	Budget	Actual	Fav. var %	Flag
Product Sales	Revenue	6,583,200	6,560,719	-0.3%	OK
Services	Revenue	2,278,800	2,140,958	-6.0%	Below target
Subscriptions	Revenue	1,772,400	1,783,720	+0.6%	OK
Cost of Goods Sold	COGS	4,810,800	4,877,106	-1.4%	OK
Salaries	Opex	2,658,600	2,645,826	+0.5%	OK
Marketing	Opex	886,200	939,177	-6.0%	Over budget
Rent	Opex	506,400	498,357	+1.6%	OK
Software	Opex	316,500	311,861	+1.5%	OK
Travel	Opex	227,880	228,247	-0.2%	OK

The scorecard reached an AUC of 0.771 on 300 held-out customers (trained on 700); the observed default rate was 12.0%. 441 customers (44.1%) fall in the High or Very High bands. Standardised coefficients show which attributes drive risk most:

Table 4. Scorecard drivers, largest magnitude first.

Feature	Std. coefficient	Effect on default risk
late_payments	+0.518	raises
debt_ratio	+0.493	raises
tenure_years	-0.365	lowers
income	-0.315	lowers
utilization	+0.026	raises

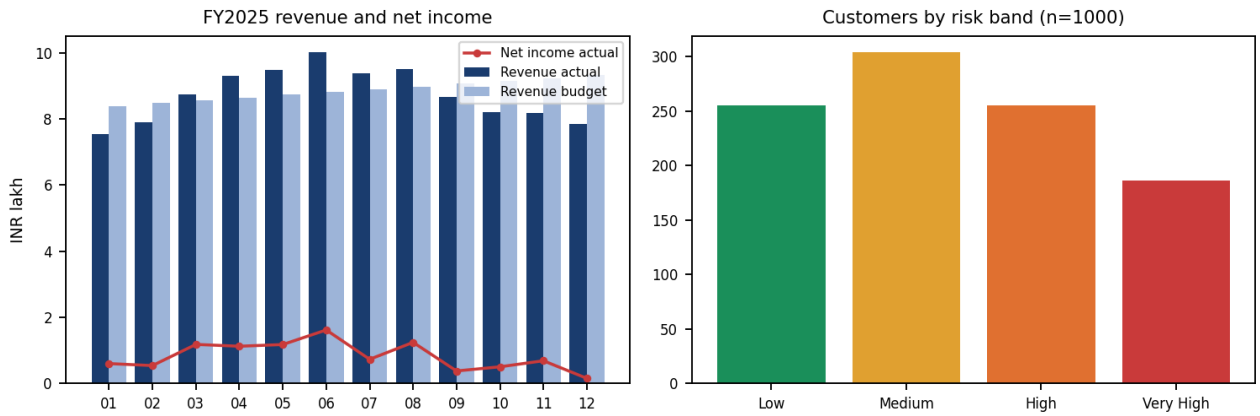


Figure 1. Monthly revenue and net income (left) and customer distribution across risk bands (right).

6. Verification

Six automated tests (pytest) cover: ledger shape and determinism under a fixed seed; the P&L identities for every month (to 1e-6); detection of both injected variance issues; scorecard quality (AUC above 0.70), score range 300 to 850 and valid band labels; monotonicity of the score in PD; and a plausible default-rate range (5% to 25%). All six pass. A bug found during development illustrates the value of the tests: the Services shortfall was initially injected for only four months, which left the annual variance below the flag threshold, and the variance test failed until the scenario was corrected.

7. Discussion and Limitations

- **Synthetic data.** All figures are generated. Absolute values carry no business meaning; the study demonstrates that the pipeline is correct and reproducible, not that the numbers describe a real firm.
- **Circularity of the AUC.** Defaults were generated from a logistic process, so a logistic regression is the correct model by construction. The reported AUC is therefore best read as a sanity check on the implementation, not as evidence of real-world predictive power, which is typically lower and must be validated on real portfolio data.
- **Estimation noise.** With a default rate near 12% on 700 training customers, only about 80 defaults inform the fit. Utilisation was generated with a positive effect, yet its fitted coefficient is close to zero (Table 4), a reminder that small samples can fail to recover true effects.
- **In-sample scoring.** Dashboard scores are computed for all customers including the training set; only the AUC uses held-out data.
- **Thresholds.** The 5% variance threshold and band cut-offs are illustrative and should be set from materiality and policy in a real deployment.
- **Scope.** The system does not implement expected-credit-loss provisioning, fairness testing of the scorecard, or multi-entity consolidation.

8. Future Work

Planned extensions are loading real ledger exports through the provided PostgreSQL schema, scheduling the pipeline monthly, adding rolling forecasts alongside budget, calibrating PD and validating the scorecard out of time, adding expected-credit-loss estimates, and generating the board pack directly as a paginated PDF.

References

[1] Siddiqi, N. (2006). Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring. Wiley.

[2] Hanley, J. A., and McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. Radiology, 143(1), 29-36.

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

“A number you cannot reproduce is only an opinion; a pipeline you can rerun is evidence.”

Mayank Swaraj

Appendix: Source Code Listings

Complete source of the system described above. Long lines are wrapped with a six-space continuation indent. The browser dashboard template (dashboard_template.html) and generated dashboard.html are delivered alongside this PDF.

Listing 1: pipeline.py

```
"""DA-04 Financial Analytics & Risk Reporting System.
Generates ledger/budget/customer data, fits a credit scorecard, builds P&L,
budget-vs-actual and a board pack, and writes dashboard.html. Run: python pipeline.py"""
import csv, json, math, os
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import StandardScaler

ROOT = os.path.dirname(os.path.abspath(__file__))
MONTHS = [f"2025-{m:02d}" for m in range(1, 13)]
ACCTS = [("Product Sales", "Revenue", 520000), ("Services", "Revenue", 180000),
         ("Subscriptions", "Revenue", 140000), ("Cost of Goods Sold", "COGS", 380000),
         ("Salaries", "Opex", 210000), ("Marketing", "Opex", 70000),
         ("Rent", "Opex", 40000), ("Software", "Opex", 25000), ("Travel", "Opex", 18000)]
FEATURES = ["income", "debt_ratio", "late_payments", "utilization", "tenure_years"]
BANDS = [("Low", 640), ("Medium", 600), ("High", 560), ("Very High", 0)]


def gen_ledger(seed=42):
    rng = np.random.default_rng(seed)
    rows = []
    for i, m in enumerate(MONTHS):
        season = 1 + 0.12 * math.sin(2 * math.pi * (i - 2) / 12)
        for a, t, b in ACCTS:
            bud = round(b * (1 + 0.01 * i), 2)
            f = season if t in ("Revenue", "COGS") else 1
            act = bud * f * (1 + rng.normal(0, 0.04))
            if a == "Marketing" and m in ("2025-07", "2025-08", "2025-09"):
                act *= 1.25  # injected overspend
            if a == "Services" and i >= 6:
                act *= 0.88  # injected shortfall
            rows.append(dict(month=m, account=a, type=t, budget=bud, actual=round(act, 2)))
    return rows


def gen_customers(n=1000, seed=7):
    r = np.random.default_rng(seed)
    income = r.lognormal(10.8, 0.4, n).round(0)
    dti = r.beta(2, 5, n).round(3)
    late = r.poisson(1.2, n)
    util = r.beta(2, 3, n).round(3)
    ten = r.integers(1, 15, n)
    z = -3.2 + 3.0 * dti + 0.45 * late + 1.8 * util - 0.08 * ten - 0.00001 * income
    d = (r.random(n) < 1 / (1 + np.exp(-z))).astype(int)
    return [dict(id=f"C{i+1:04d}", income=income[i], debt_ratio=dti[i], late_payments=int(late[i]),
                 utilization=util[i], tenure_years=int(ten[i]), defaulted=int(d[i])) for i in range(n)]


def to_score(pd_):
    pd_ = np.clip(pd_, 1e-4, 1 - 1e-4)
    s = 650 + (40 / math.log(2)) * np.log((1 - pd_) / pd_ / 20)
    return np.clip(np.round(s), 300, 850).astype(int)


def band(s):
    return next(b for b, lo in BANDS if s >= lo)


def fit_scorecard(cust, seed=1):
    X = np.array([[c[f] for f in FEATURES] for c in cust], float)
    y = np.array([c["defaulted"] for c in cust])
    idx = np.random.default_rng(seed).permutation(len(y))
    cut = int(0.7 * len(y))
    tr, te = idx[:cut], idx[cut:]
    sc = StandardScaler().fit(X[tr])
    m = LogisticRegression(max_iter=1000).fit(sc.transform(X[tr]), y[tr])
    auc = roc_auc_score(y[te], m.predict_proba(sc.transform(X[te]))[:, 1])
    pdv = m.predict_proba(sc.transform(X))[:, 1]
    scores = to_score(pdv)
    for c, p, s in zip(cust, pdv, scores):
        c.update(pd=round(float(p), 4), score=int(s), band=band(int(s)))
    return dict(auc=round(float(auc), 3), coefs={f: round(float(w), 3) for f, w in zip(FEATURES, m.coef_[0])},
                train=int(cut), test=int(len(y) - cut))


def pnl(ledger):
    out = []
    for m in MONTHS:
        r = {"month": m}
```

```
        for k in ("actual", "budget"):
            g = lambda t: sum(x[k] for x in ledger if x["month"] == m and x["type"] == t)
            rev, cogs, opex = g("Revenue"), g("COGS"), g("Opex")
            r[k] = dict(revenue=rev, cogs=cogs, gross=rev - cogs, opex=opex, net=rev - cogs - opex)
        out.append(r)
    return out


def budget_vs_actual(ledger):
    out = []
    for a, t, _ in ACCTS:
        bud = sum(x["budget"] for x in ledger if x["account"] == a)
        act = sum(x["actual"] for x in ledger if x["account"] == a)
        fav = (act - bud) if t == "Revenue" else (bud - act)
        pct = fav / bud * 100
        out.append(dict(account=a, type=t, budget=bud, actual=act, fav=fav, fav_pct=pct,
                        flag="Over budget" if (t != "Revenue" and pct < -5) else
                             "Below target" if (t == "Revenue" and pct < -5) else "OK"))
    return out


def narrative(pl, bva, cust, model):
    A = lambda k: sum(r["actual"][k] for r in pl)
    B = lambda k: sum(r["budget"][k] for r in pl)
    n = []
    n.append(f"Revenue for FY2025 was {A('revenue'):,.0f} against a budget of {B('revenue'):,.0f}
      ({(A('revenue')/B('revenue')-1)*100:+.1f}%).")
    n.append(f"Net income was {A('net'):,.0f} versus a budgeted {B('net'):,.0f} ({(A('net')/B('net')-1)*100:+.1f}%).")
    bad = sorted([b for b in bva if b["flag"] != "OK"], key=lambda b: b["fav_pct"])[:3]
    if bad:
        n.append("Attention items: " + "; ".join(f"{b['account']} ({b['fav_pct']:+.1f}% vs budget,
      {b['flag'].lower()})" for b in bad) + ".")
    hi = sum(c["band"] in ("High", "Very High") for c in cust) / len(cust) * 100
    dr = sum(c["defaulted"] for c in cust) / len(cust) * 100
    n.append(f"Credit book: {len(cust)} customers, observed default rate {dr:.1f}%, {hi:.1f}% in High/Very High risk
      bands. Scorecard AUC on hold-out: {model['auc']}.")
    return n


def write_csv(path, rows):
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)


def build():
    led, cust = gen_ledger(), gen_customers()
    model = fit_scorecard(cust)
    pl, bva = pnl(led), budget_vs_actual(led)
    write_csv(f"{ROOT}/data/gl_ledger.csv", led)
    write_csv(f"{ROOT}/data/customers_scored.csv", cust)
    bands = {b: sum(c["band"] == b for c in cust) for b, _ in BANDS}
    data = dict(months=MONTHS, pl=pl, bva=bva, model=model, bands=bands, narrative=narrative(pl, bva, cust, model),
                top=sorted(cust, key=lambda c: c["score"])[:10], n=len(cust),
                default_rate=round(sum(c["defaulted"] for c in cust) / len(cust) * 100, 2))
    json.dump(dict(auc=model["auc"], bands=bands, default_rate=data["default_rate"]),
      open(f"{ROOT}/docs/metrics.json", "w"), indent=1)
    html = open(f"{ROOT}/dashboard_template.html", encoding="utf-8").read().replace("__DATA__", json.dumps(data))
    open(f"{ROOT}/dashboard.html", "w", encoding="utf-8").write(html)
    return data


if __name__ == "__main__":
    d = build()
    print("AUC", d["model"]["auc"], "| bands", d["bands"], "| dashboard.html written")
```

Listing 2: tests/test_pipeline.py

```
import os, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import pipeline as p

def test_ledger_shape_and_determinism():
    a, b = p.gen_ledger(), p.gen_ledger()
    assert len(a) == 12 * len(p.ACCTS) and a == b

def test_pnl_identity():
    for r in p.pnl(p.gen_ledger()):
        for k in ("actual", "budget"):
            x = r[k]
            assert abs(x["net"] - (x["revenue"] - x["cogs"] - x["opex"])) < 1e-6
            assert abs(x["gross"] - (x["revenue"] - x["cogs"])) < 1e-6

def test_variance_flags_injected_issues():
    flags = {b["account"]: b["flag"] for b in p.budget_vs_actual(p.gen_ledger())}
    assert flags["Marketing"] == "Over budget" and flags["Services"] == "Below target"

def test_scorecard_quality_and_ranges():
    c = p.gen_customers(); m = p.fit_scorecard(c)
    assert m["auc"] > 0.70
    assert all(300 <= x["score"] <= 850 for x in c)
    assert {x["band"] for x in c} <= {b for b, _ in p.BANDS}
```

```
def test_score_monotonic():
    assert p.to_score([0.02])[0] > p.to_score([0.30])[0]

def test_default_rate_sane():
    c = p.gen_customers(); r = sum(x["defaulted"] for x in c) / len(c)
    assert 0.05 < r < 0.25
```

Listing 3: schema.sql

```
-- PostgreSQL schema (load CSVs from data/ with \copy)
CREATE TABLE gl_ledger (month CHAR(7), account TEXT, type TEXT CHECK (type IN ('Revenue','COGS','Opex')), budget
      NUMERIC(14,2), actual NUMERIC(14,2));
CREATE TABLE customers_scored (id TEXT PRIMARY KEY, income NUMERIC, debt_ratio NUMERIC, late_payments INT, utilization
       NUMERIC, tenure_years INT, defaulted SMALLINT, pd NUMERIC(6,4), score INT, band TEXT);
```

Listing 4: build_paper.py

```
"""Builds DA-04_Research_Paper_and_Source.pdf; all numbers come from the live pipeline run."""
import os, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from reportlab.lib.units import mm
import paperkit as k, pipeline as p


R = p.ROOT
d = p.build()
pl, bva, m, bands = d["pl"], d["bva"], d["model"], d["bands"]
A = lambda x: sum(r["actual"][x] for r in pl); B = lambda x: sum(r["budget"][x] for r in pl)
f0 = lambda x: f"{x:,.0f}"; pc = lambda a, b: f"{(a / b - 1) * 100:+.1f}%"
os.makedirs("/tmp/fig4", exist_ok=True)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.1)); mo = [x[5:] for x in d["months"]]
ax[0].bar([i - .2 for i in range(12)], [r["actual"]["revenue"] / 1e5 for r in pl], .4, label="Revenue actual",
      color="#1a3c6e")
ax[0].bar([i + .2 for i in range(12)], [r["budget"]["revenue"] / 1e5 for r in pl], .4, label="Revenue budget",
      color="#9db4d8")
ax[0].plot(range(12), [r["actual"]["net"] / 1e5 for r in pl], "o-", color="#c93a3a", label="Net income actual", ms=3)
ax[0].set_xticks(range(12)); ax[0].set_xticklabels(mo, fontsize=7); ax[0].set_ylabel("INR lakh", fontsize=8);
      ax[0].legend(fontsize=6.5); ax[0].set_title("FY2025 revenue and net income", fontsize=9)
ax[1].bar(list(bands), list(bands.values()), color=["#1a8f5a", "#e0a030", "#e07030", "#c93a3a"]);
      ax[1].set_title("Customers by risk band (n=%d)" % d["n"], fontsize=9)
for a in ax: a.tick_params(labelsize=7)
plt.tight_layout(); plt.savefig("/tmp/fig4/f1.png", dpi=170); plt.close()

coefs = sorted(m["coefs"].items(), key=lambda x: -abs(x[1]))
bad = [b for b in bva if b["flag"] != "OK"]
hi = bands["High"] + bands["Very High"]
sections = [
 ("1. Introduction", [
  "Month-end finance reporting in small and mid-sized firms is usually assembled by hand from ledger exports and
      spreadsheets. The process is slow, hard to audit and prone to copy errors, and the credit-risk view of the
      customer book is often produced separately, if at all. This paper describes a compact, fully reproducible system
       that automates four outputs from one pipeline: a monthly profit and loss (P&L) statement, a budget-versus-
      actual variance analysis with automatic exception flags, a scorecard-based credit risk rating for every
      customer, and a board-pack narrative that summarises the position in plain language.",
  "The contributions are: (i) an end-to-end pipeline, from data to an offline dashboard, that can be regenerated with
      one command; (ii) a transparent logistic-regression scorecard that converts probability of default (PD) into a
      300 to 850 points scale with named risk bands; (iii) a rule-based variance engine whose flags were verified
      against deliberately injected issues; and (iv) an automated test suite that checks accounting identities,
      determinism, model quality and flag behaviour."]),
 ("2. Background and Related Work", [
  "Credit scorecards built on logistic regression remain a standard industry tool because the coefficients are
      directly interpretable and the points scale is easy to explain to credit officers and auditors [1]. Model
      discrimination is commonly summarised by the area under the ROC curve (AUC), which equals the probability that a
       randomly chosen defaulter receives a higher PD than a randomly chosen non-defaulter [2]. Points-to-double-odds
      (PDO) scaling, used here, maps log-odds linearly to points so that a fixed number of points always corresponds
      to a doubling of good-to-bad odds.",
  "Budget variance analysis compares actual results with plan for each account and reports the difference by sign
      convention: for revenue, actual above budget is favourable; for costs, actual below budget is favourable.
      Automating the rule is simple, but its value depends on a sensible exception threshold and on correct handling
      of the sign convention, both of which are made explicit and tested in this work."]),
 ("3. System Design", [
  "The pipeline (pipeline.py) has five stages: data generation, P&L computation, variance analysis, scorecard fitting
      and scoring, and report generation. Outputs are two CSV files (gl_ledger.csv, customers_scored.csv), a metrics
      file, and a single self-contained HTML dashboard that embeds its data and draws charts with canvas, so it opens
      offline in any browser. A PostgreSQL schema (schema.sql) is provided for loading the CSVs into a database.",
  ("table", [["Table", "Columns"], ["gl_ledger", "month, account, type (Revenue/COGS/Opex), budget, actual"],
             ["customers_scored", "id, income, debt_ratio, late_payments, utilization, tenure_years, defaulted, pd,
      score, band"]], "Table 1. Data model.", [35 * mm, 139 * mm])]),
 ("4. Methods", [
  ("h2", "4.1 Ledger simulation"),
  "No real company ledger was available, so a synthetic general ledger was generated with a fixed random seed. Nine
      accounts across three types (three revenue, one cost of goods sold, five operating expense) are simulated for
      the twelve months of 2025. The monthly budget for each account is its base value grown by 1% per month. Actuals
      equal budget multiplied by a seasonal factor (applied to revenue and COGS) and Gaussian noise:",
  ("eq", "actual = budget * season * (1 + e),  season = 1 + 0.12 * sin(2*pi*(i-2)/12),  e ~ N(0, 0.04)"),
  "Two issues were injected on purpose so that detection can be verified rather than assumed: Marketing spend is
      raised by 25% in July to September, and Services revenue is reduced by 12% from July to December.",
  ("h2", "4.2 P&L and variance rules"),
  "For every month, gross profit = revenue - COGS and net income = gross profit - operating expenses, for both actual
```

```
      and budget. For each account the favourable variance is (actual - budget) for revenue and (budget - actual) for
      costs, expressed as a percentage of budget. An account is flagged \"Over budget\" (cost) or \"Below target\"
      (revenue) when the favourable variance is below -5%.",
  ("h2", "4.3 Credit scorecard"),
  "A population of 1,000 customers is simulated with five features: income (log-normal), debt ratio (Beta(2,5)), late
      payments (Poisson(1.2)), credit utilisation (Beta(2,3)) and tenure in years (uniform 1 to 14). Defaults are
      drawn from a Bernoulli distribution with probability sigmoid(z), where",
  ("eq", "z = -3.2 + 3.0*debt_ratio + 0.45*late_payments + 1.8*utilization - 0.08*tenure - 0.00001*income"),
  "The data are split 70/30 by a seeded permutation. Features are standardised using the training set and an
      L2-regularised logistic regression (scikit-learn defaults, 1,000 iterations) is fitted. AUC is computed on the
      held-out 30%. Every customer is then scored in-sample for the dashboard using the PDO transformation",
  ("eq", "score = 650 + (40 / ln 2) * ln( (1 - PD) / PD / 20 ),  clipped to [300, 850]"),
  "which gives 650 points at 20:1 good-to-bad odds and 40 points per doubling of odds. Bands are Low (640 and above),
      Medium (600 to 639), High (560 to 599) and Very High (below 560); the cut-offs were chosen after inspecting the
      score distribution so that all four bands are populated.",
  ("h2", "4.4 Board-pack narrative"),
  "A short narrative is generated from the computed figures: full-year revenue and net income against budget, the
      three worst flagged accounts, and a credit-book summary. Because every sentence is produced from computed
      values, the text cannot drift from the numbers."]),
 ("5. Results", [
  f"Full-year revenue was INR {f0(A('revenue'))} against a budget of INR {f0(B('revenue'))} ({pc(A('revenue'),
      B('revenue'))}). Net income was INR {f0(A('net'))} against INR {f0(B('net'))} ({pc(A('net'), B('net'))}).",
  ("table", [["Line", "Actual (INR)", "Budget (INR)", "Variance"]] + [[n, f0(A(x)), f0(B(x)), pc(A(x), B(x))] for n, x
       in [("Revenue", "revenue"), ("COGS", "cogs"), ("Gross profit", "gross"), ("Operating expenses", "opex"), ("Net
      income", "net")]], "Table 2. FY2025 P&L, actual versus budget.", [50 * mm, 42 * mm, 42 * mm, 40 * mm]),
  f"The variance engine flagged {len(bad)} account(s): " + "; ".join(f"{b['account']} ({b['fav_pct']:+.1f}%,
      {b['flag'].lower()})" for b in bad) + ". These are exactly the two accounts into which issues were injected, and
       no other account crossed the 5% threshold.",
  ("table", [["Account", "Type", "Budget", "Actual", "Fav. var %", "Flag"]] + [[b["account"], b["type"],
      f0(b["budget"]), f0(b["actual"]), f"{b['fav_pct']:+.1f}%", b["flag"]] for b in bva], "Table 3. Budget versus
      actual by account (full year).", [38 * mm, 20 * mm, 30 * mm, 30 * mm, 24 * mm, 32 * mm]),
  f"The scorecard reached an AUC of {m['auc']} on {m['test']} held-out customers (trained on {m['train']}); the
      observed default rate was {d['default_rate']}%. {hi} customers ({hi / d['n'] * 100:.1f}%) fall in the High or
      Very High bands. Standardised coefficients show which attributes drive risk most:",
  ("table", [["Feature", "Std. coefficient", "Effect on default risk"]] + [[a, f"{b:+.3f}", "raises" if b > 0 else
      "lowers"] for a, b in coefs], "Table 4. Scorecard drivers, largest magnitude first.", [50 * mm, 40 * mm, 60 *
      mm]),
  ("fig", "/tmp/fig4/f1.png", "Figure 1. Monthly revenue and net income (left) and customer distribution across risk
      bands (right).", 170, 0.345)]),
 ("6. Verification", [
  "Six automated tests (pytest) cover: ledger shape and determinism under a fixed seed; the P&L identities for every
      month (to 1e-6); detection of both injected variance issues; scorecard quality (AUC above 0.70), score range 300
       to 850 and valid band labels; monotonicity of the score in PD; and a plausible default-rate range (5% to 25%).
      All six pass. A bug found during development illustrates the value of the tests: the Services shortfall was
      initially injected for only four months, which left the annual variance below the flag threshold, and the
      variance test failed until the scenario was corrected."]),
 ("7. Discussion and Limitations", [
  ("bul", ["<b>Synthetic data.</b> All figures are generated. Absolute values carry no business meaning; the study
      demonstrates that the pipeline is correct and reproducible, not that the numbers describe a real firm.",
           "<b>Circularity of the AUC.</b> Defaults were generated from a logistic process, so a logistic regression
      is the correct model by construction. The reported AUC is therefore best read as a sanity check on the
      implementation, not as evidence of real-world predictive power, which is typically lower and must be validated
      on real portfolio data.",
           "<b>Estimation noise.</b> With a default rate near 12% on 700 training customers, only about 80 defaults
      inform the fit. Utilisation was generated with a positive effect, yet its fitted coefficient is close to zero
      (Table 4), a reminder that small samples can fail to recover true effects.",
           "<b>In-sample scoring.</b> Dashboard scores are computed for all customers including the training set; only
       the AUC uses held-out data.",
           "<b>Thresholds.</b> The 5% variance threshold and band cut-offs are illustrative and should be set from
      materiality and policy in a real deployment.",
           "<b>Scope.</b> The system does not implement expected-credit-loss provisioning, fairness testing of the
      scorecard, or multi-entity consolidation."]),
 ]),
 ("8. Future Work", [
  "Planned extensions are loading real ledger exports through the provided PostgreSQL schema, scheduling the pipeline
      monthly, adding rolling forecasts alongside budget, calibrating PD and validating the scorecard out of time,
      adding expected-credit-loss estimates, and generating the board pack directly as a paginated PDF."]),
]
refs = ["[1] Siddiqi, N. (2006). Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring.
      Wiley.",
        "[2] Hanley, J. A., and McNeil, B. J. (1982). The meaning and use of the area under a receiver operating
      characteristic (ROC) curve. Radiology, 143(1), 29-36.",
        "[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning
      Research, 12, 2825-2830."]
summary = [f"<b>Problem:</b> manual month-end finance and credit-risk reporting is slow, error-prone and hard to
      audit.",
           f"<b>Approach:</b> one reproducible pipeline producing P&L, budget-vs-actual with exception flags, a
      logistic scorecard (300-850 points, four bands) and an auto-written board pack, delivered as an offline HTML
      dashboard.",
           f"<b>Results:</b> FY2025 revenue INR {f0(A('revenue'))} ({pc(A('revenue'), B('revenue'))} vs budget); net
      income {pc(A('net'), B('net'))} vs budget; {len(bad)} accounts flagged, matching both injected issues; scorecard
       hold-out AUC {m['auc']}; 6 of 6 tests pass.",
           "<b>Caveat:</b> data are synthetic, so AUC validates the implementation rather than real-world predictive
      power."]
k.build(f"{R}/DA-04_Research_Paper_and_Source.pdf", "Automated Financial Analytics and Credit Risk Reporting", "A
      reproducible pipeline for P&L, budget variance and scorecard-based risk banding (DA-04)",
        "DA-04 Financial Analytics and Risk Reporting System", summary, sections, refs,
        "A number you cannot reproduce is only an opinion; a pipeline you can rerun is evidence.",
        [("pipeline.py", f"{R}/pipeline.py"), ("tests/test_pipeline.py", f"{R}/tests/test_pipeline.py"),
      ("schema.sql", f"{R}/schema.sql"), ("build_paper.py", f"{R}/build_paper.py")])
print("pdf built")
```